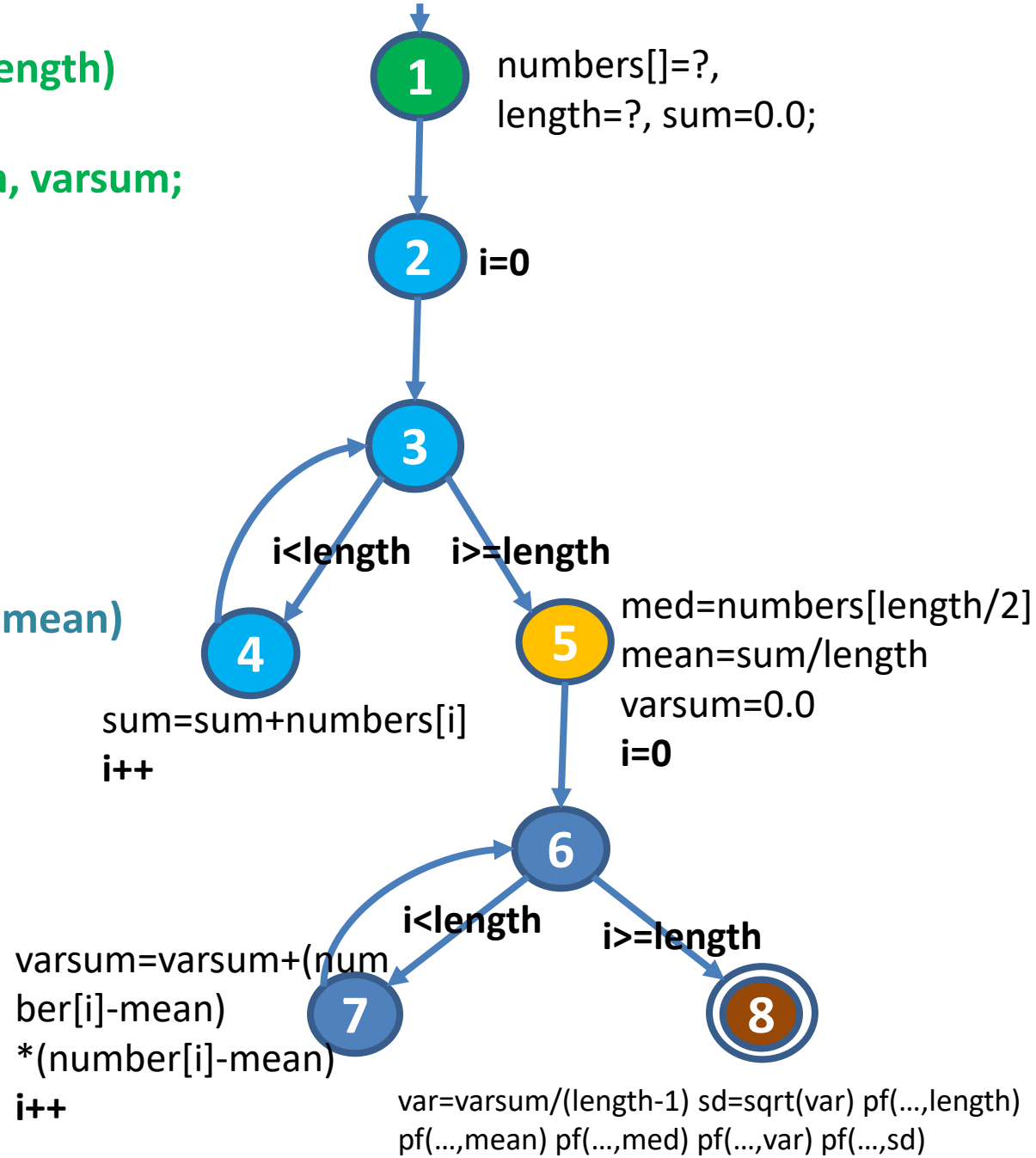
Coding and Testing, part-II

S. S Satapathy

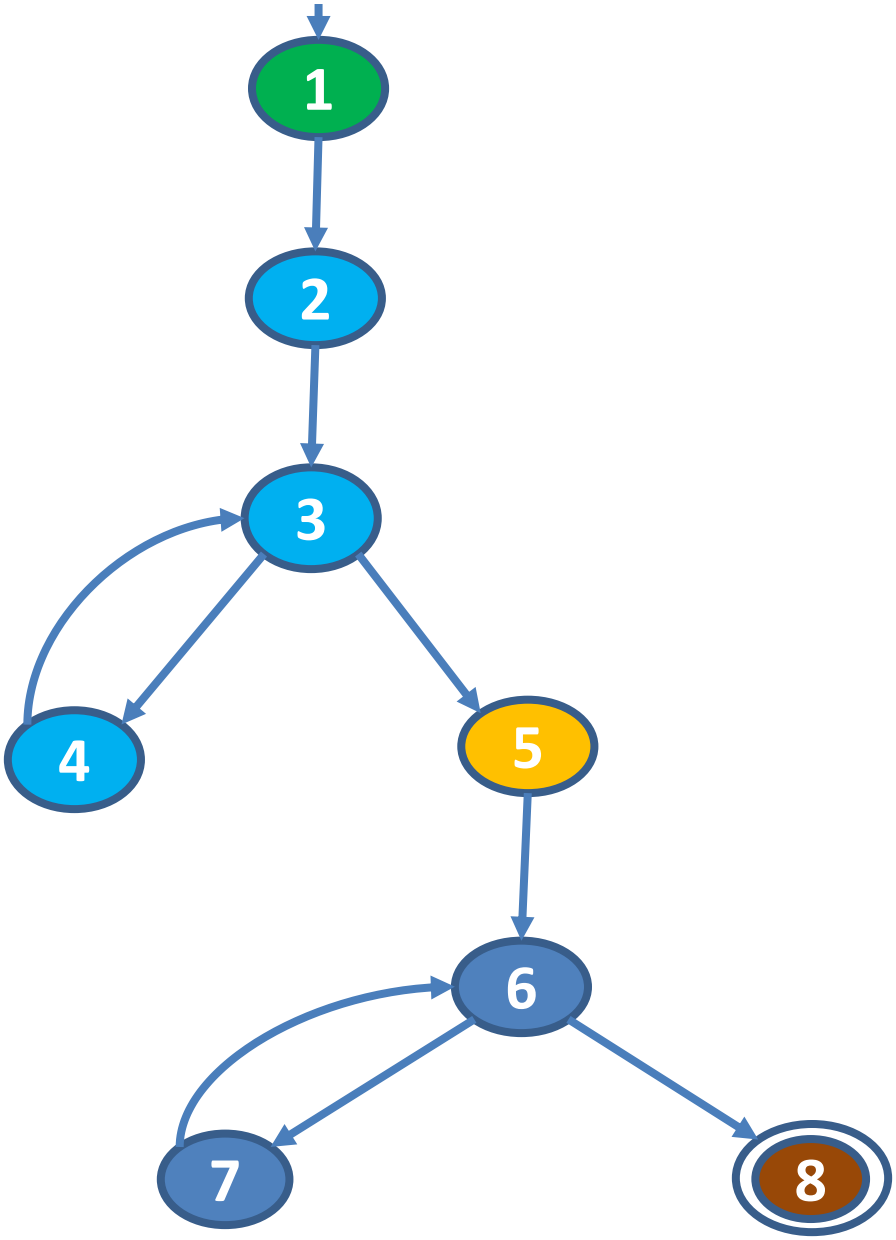
Graphical Representation of a Program using Control Flow Graph (CFG)

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%lf",length);
  printf("Mean=%lf",mean);
  printf("Median=%lf",med);
  printf("Variance=%lf",var);
  printf("Std. dev=%lf",sd);
}
```



Control Flow Graph

During execution of any program, we traverse the graph visiting nodes and edges of the corresponding control flow graph.

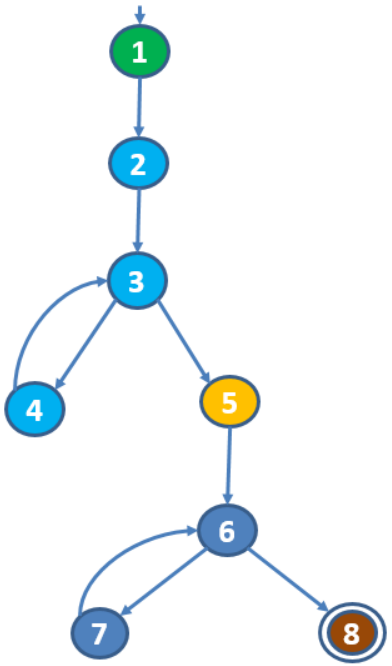


Path in a graph

- **Path p** is **a sequence of vertices** $v_1, v_2, \ldots, v_n$ such that

$$(v_i, v_{i+1}) \in E \ for 1 \le i \le (n-1)$$

- **length** of the path is a **number of edges**; **single vertex** could be **a paths** and of the **length 0**
- A **sub-path of a path** is just a **subsequences of vertices** that occur **in the path**.

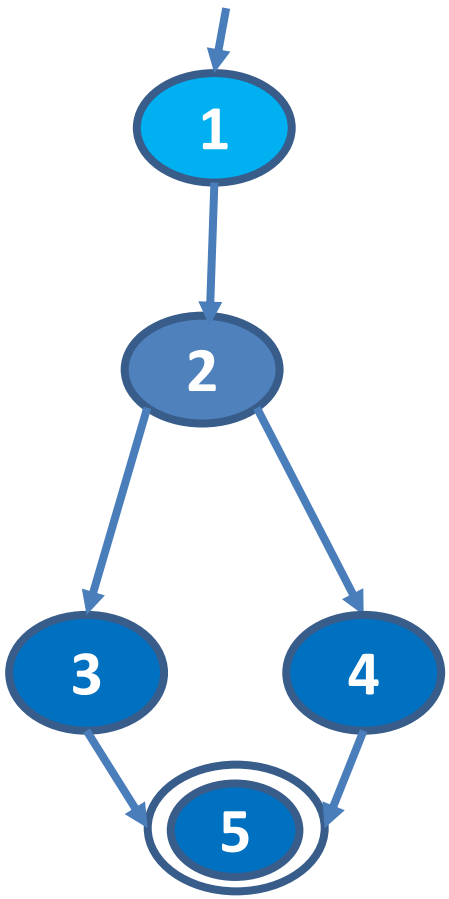


(1,2,3,4) is a path of length 3
(3,5,6) is a path of length 2
(2,3,4,3,4) is a path of length 4
(2,3,4,5) is not a path

(4,3,4) is a **sub-path** of path (2,3,4,3,4)

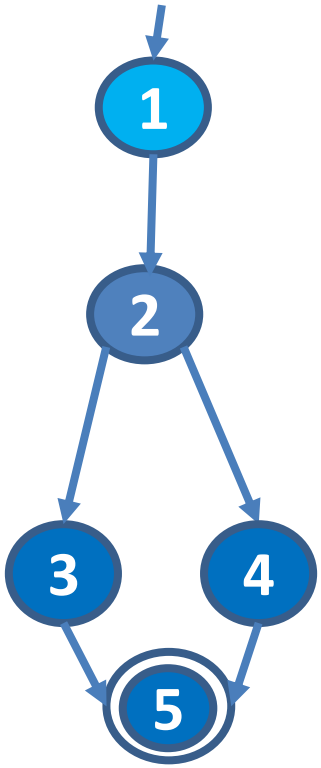
Reachability in a graph

- **A node u is reachable** in a graph G, **if there is a path from the initial node** to **the node u** in graph G
- **An edge e=(u,v) is reachable** in a graph G, **if there is a path** from the **initial node to the edge e** in graph G
- **A sub graph G'** of G **is reachable** in a graph G, **if any one of the node in the sub graph G' is reachable** is from the initial node of G.



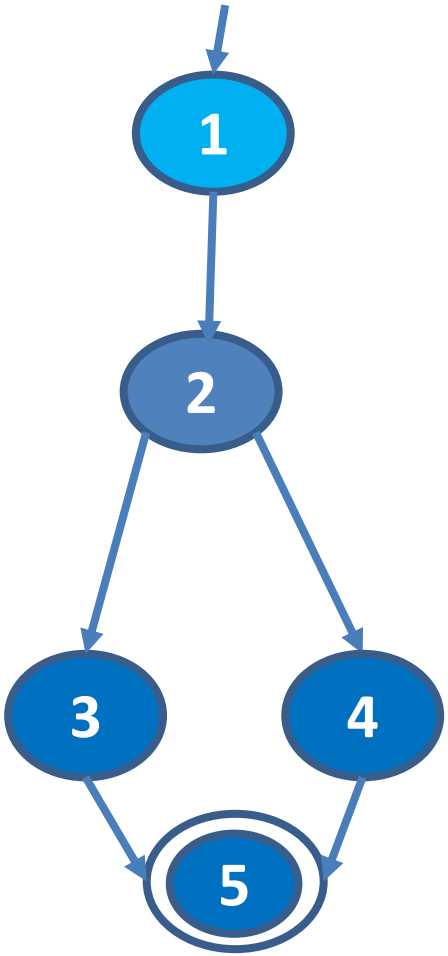
Visiting and Touring in a graph

- A **test path p** **visits a vertex v**, if **v** **occurs along the path p**. Similarly, **a test path p** **visits an edge e** if **e** **occurs along the path p**.
- A **test path p** **tours path q**, if **q** happens to be a **sub path** of **p**.
- Since each **edge/ vertex** is a **sub path**, **test path tours an edge/vertex**.
- Consider the **path (1,2,4,5)**. It **visits** the nodes **1, 2 and 4**. It also **visits** edges **(2,4)** and **(4,5)**. It **tours** sub-path **(2,4,5)**.



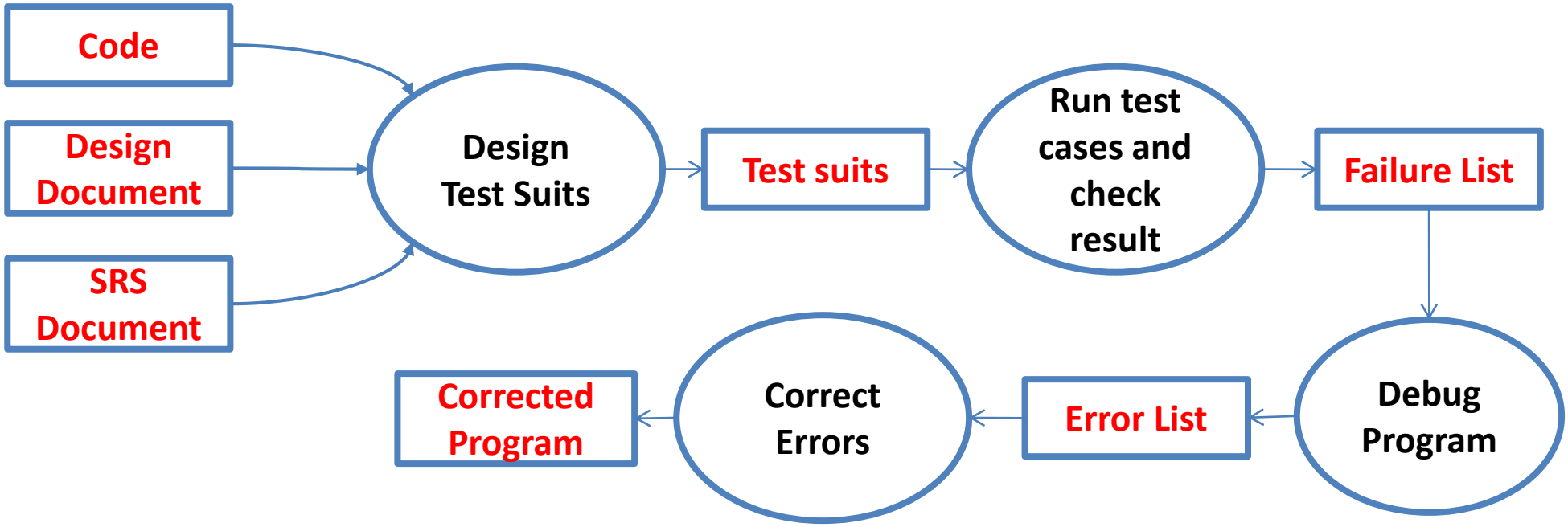
Test path in a graph

- A **test path** is **a path** in a graph **that starts at the first node** and **end in the end node**
- **Test path represents execution of a test case**
  - **Some test paths** can be traversed to **represent execution of test cases (feasible test paths)**
  - **Some paths can't be executed (infeasible test paths)**



Program Testing

- **Testing a program** consists of **providing** the program with a set of **test inputs** (or test cases) and **observing if the program behaves as expected**. **If** the program **fails to behave as expected**, then the **conditions under which failure occurs** are **noted** for later **debugging and correction**.



Design of Test Cases

- **Exhaustive testing of almost any system is practically difficult :** Input Domain is extremely large or infinite
- **An optimal test suite** (not a random large suit) that can uncover as many errors as possible should be designed
- Let us have the **wrongly** code to find maximum number

```
If (x>y)        max = x;
else            max = x;
```

- Test suite, **{(x=3,y=2), (x=2,y=3)} can detect the error**
- Whereas a larger test suite **{(x=3,y=2), (x=4,y=3), (x=5,y=1)} can't detect the error**

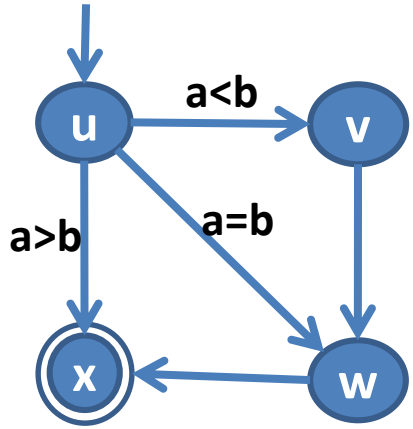
Two different Criteria of Graph Coverage

We can utilize features of a CFG in two different ways for developing test cases with help of test paths.

1. **Structural Coverage Criteria:**
   - **Defined on a graph only in terms of the nodes and edges**
2. **Data flow Coverage criteria:**
   - Requires a **graph** to be **annotated with reference to** the variables and **defines criteria requirements based on the annotations**

Test and Test Path

- When we run a program with certain input, we visit a sequence of nodes in the CFG. **In other words, when a test case t executes a path**, we call it the test path executed by t: **path(t)**
- Similarly when a **set of test cases T execute a set of path**, we denote them as: **path(T)**



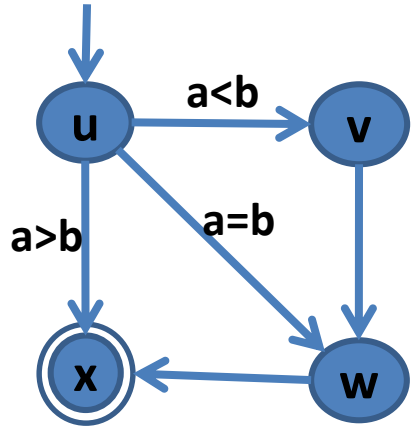
Test case input: (a=0,b=1) Test path:u,v,w,x
Test case input: (a=1,b=1) Test path:u,w,x
Test case input: (a=3,b=1) Test path:u,x

Structural Coverage Criteria

- Node/Vertex coverage
- Edge coverage
- Edge pair coverage
- Path coverage
  - Complete path coverage
  - Prime path coverage
  - Complete round trip coverage
  - Simple round trip coverage

Graph Coverage Criteria

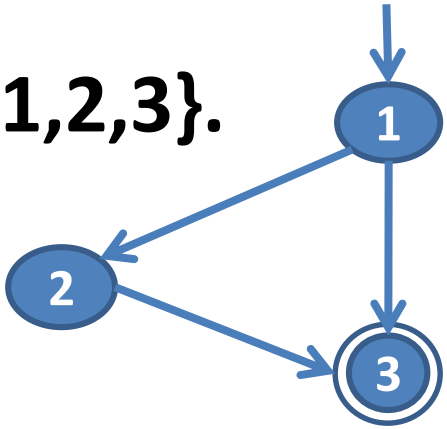
- **Test requirement(TR): Describe** the **properties** of the **test paths**
- **Test Criteria(C):** are the **rules** that **defines** the **test requirements**
- **Satisfaction:** Given a set of **TR** (test requirements), **for a criteria C, a set of tests T satisfies C** on a graph
  - Iff for every test requirement t in TR, there is a test path in path (T) that meet the test requirement.
  - **For example,** the below given test cases satisfies **branch coverage at node *u*.**



Test case input: (a=0,b=1) Test path:u,v,w,x
Test case input: (a=1,b=1) Test path:u,w,x
Test case input: (a=3,b=1) Test path:u,x

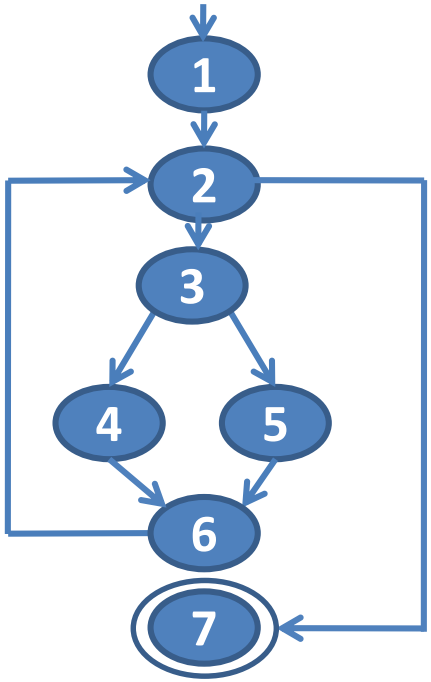
Node Coverage

- **Node coverage:** Test requirement (**TR**) is that the **test cases visit each node** in the graph
- **Test set T** satisfies node coverage on a Graph G
  - **Iff for every syntactically reachable node *n* in G, there is some path *p* in path(T) such that *p* visits *n*.**
- Simply we can say that TR contains each reachable node from the start node in G
- **Node coverage: TR = {1,2,3}, test path={1,2,3}.**



Node (Statement) coverage example

```
//Compute GCD of two numbers
int compute_gcd(int x, int y)
{
        while (x! = y)
        {
                if (x>y)
                x= x - y;
                else y = y - x;
        }
        return x;
}
```

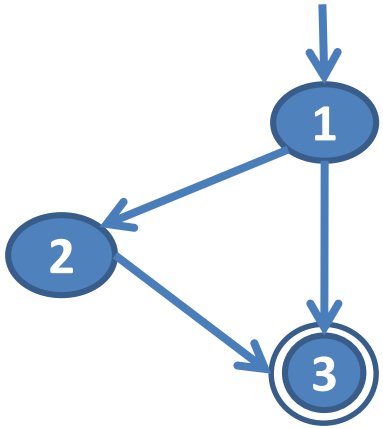


- In order to design the **test cases for node (statement) coverage** in this example, **conditional expression in the while statement must be true** and **if statement must be both true and false in turn**.
- So one possible test suit can be **{(x=4, y=3), (x=3, y=5)}**

Edge Coverage

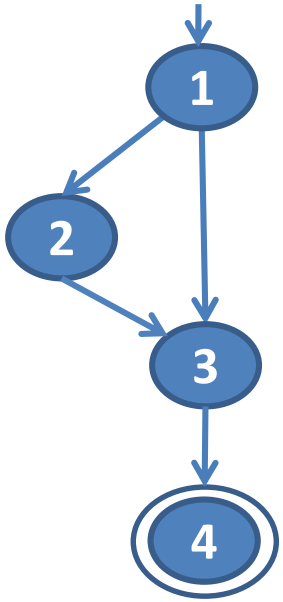
- **Edge coverage: TR** contains **each reachable path of length up to 1**.
  - It includes paths of length 0 and 1.
  - It allows edge coverage for graphs with one node and no edges.
  - Allowing length up to 1 allows **edge coverage to subsume (include) node coverage i.e. edge coverage is stronger than node coverage**
- **Edge coverage: TR = {(1,2),(1,3),(2,3)},**

  test path={(1,2,3),(1,3)}.



Edge coverage example

```
int function(int x, int y)
{
        if(x! = y){
                printf("OK")
        }
        return 0;
}
```

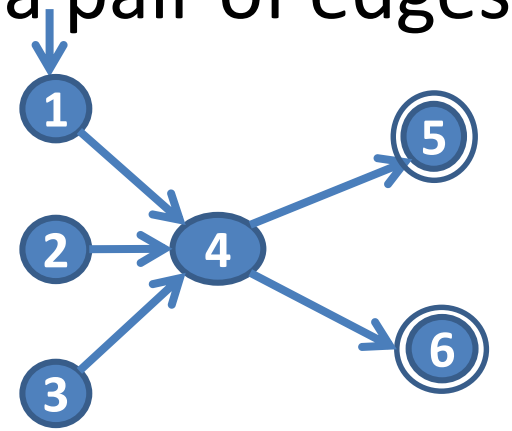


- So one possible test suit based on **edge coverage can be {(x=3, y=4), (x=3, y=3)}**
- Where as **for node coverage only {(x=3, y=4)}** could have suffice.
- **Node and Edge coverage are same for linear sequence of statements.**

Covering Multiple Edges: Edge-Pair Coverage

- **Number of test cases for node/edge coverage** might be **too large** for any real software as corresponding graph can be quite large.
- Therefore **Edge-pair coverage might be more suitable**.
- **Edge-Pair Coverage(EPC): TR** contains **each reachable path of length up to 2**, inclusive in G.
- Path of length up to 2 corresponds to a pair of edges.
- **TR = {(1,4,5), (2,4,5), (3,4,5), (1,4,6), (2,4,6), (3,4,6)}**

test path=Same as above set.



Covering Multiple Edges contd…

- An extension of Edge-pair coverage is to consider **all paths**.
- **Complete path Coverage: TR** contains **all paths** in G.
  - Keeping presence of loops in view, **number of paths in any real CGF can be infinity** and therefore finding test paths for complete path coverage is impossible.
  - Therefore it may not be a useful test requirement
- **Specified Path coverage:** TR contains a set S of paths, where S is specified by the user/tester.

Examples on Different Coverage Criteria

- **Node Coverage:**

  TR= ?, Test paths: ?

- **Edge coverage:**

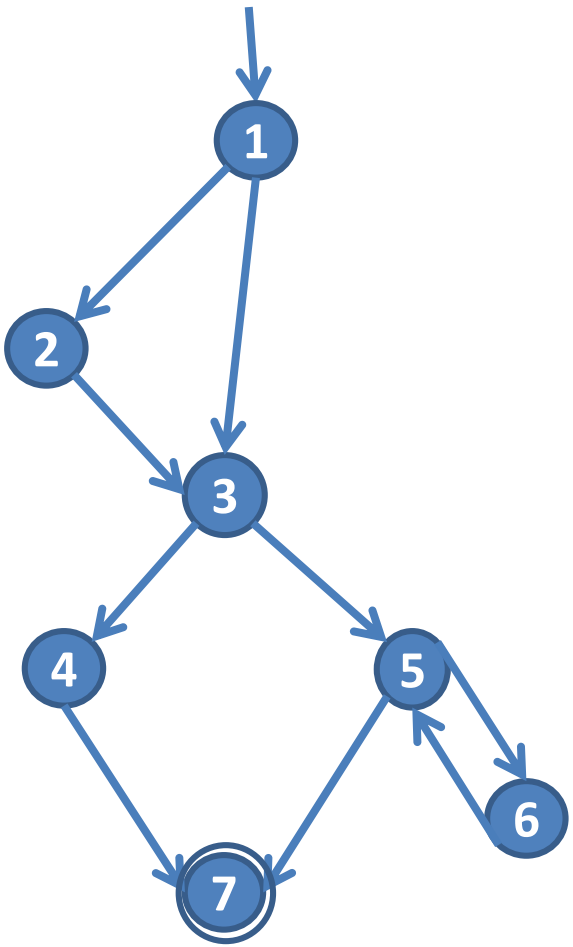
  TR= ?, Test paths: ?

- **Edge pair coverage:**

  TR= ?, Test paths: ?

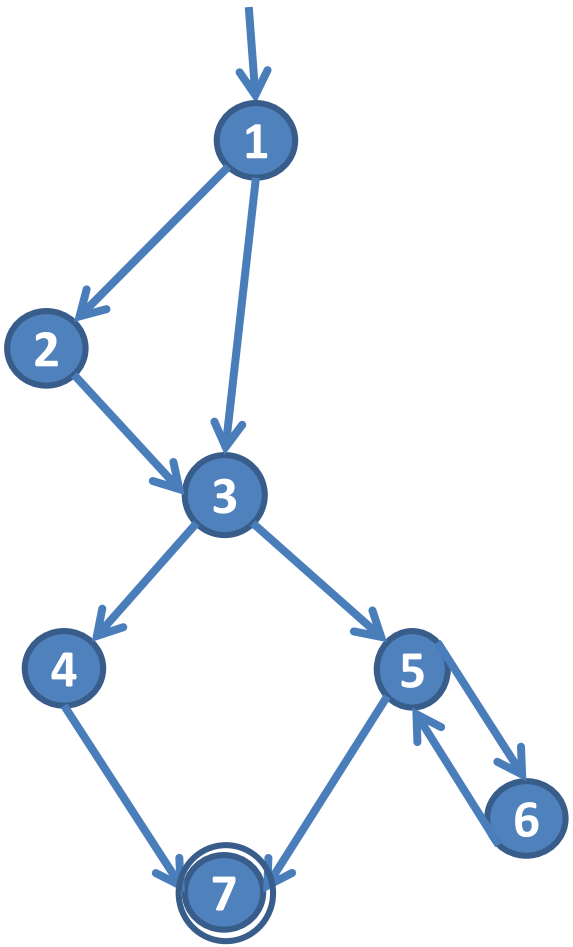
- **Complete coverage:**

  TR= ?, Test paths: ?



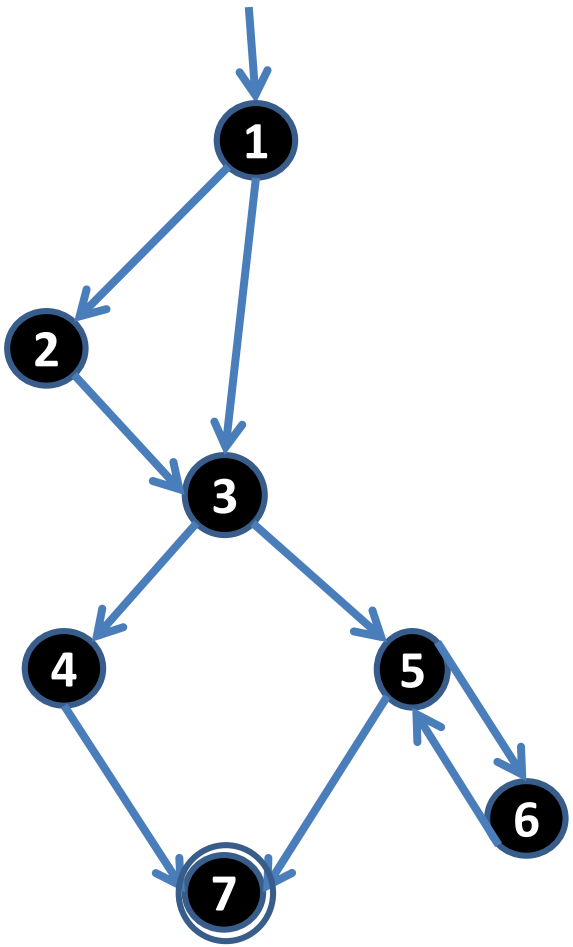
Different Coverage Criteria Example

- **Node Coverage:** TR={1,2,3,4,5,6,7}, Test paths: ?
- **Edge coverage:** TR={(1,2), (1,3), (2,3), (3,4), (3,5), (5,6), (6,5), (4,7), (5,7)}, Test paths: ?
- **Edge pair coverage:** TR={(1,2,3), (1,3,4), (1,3,5), …}, Test paths: ?
- **Complete coverage:** TR={(1,2,3,4,7), (1,2,3,5,7), (1,2,3,5,6,5,7), …so on}, Test paths: ?



Different Coverage Criteria Example

- **Node Coverage:** TR={1,2,3,4,5,6,7}, Test paths: {(1,2,3,4,7),(1,3,5,6,5,7)}
- **Edge coverage:** TR={(1,2), (1,3), (2,3), (3,4), (3,5), (5,6), (6,5), (4,7), (5,7)}, Test paths: {(1,2,3,4,7),(1,3,5,6,5,7)}
- **Edge pair coverage:** TR={(1,2,3), (1,3,4), (1,3,5), …}, Test paths: {(1,2,3,4,7),(1,3,5,7),(1,3,4,7), (1,3,5,6,5,6,5,7)}
- **Complete coverage:** TR={(1,2,3,4,7), (1,2,3,5,7), (1,2,3,5,6,5,7), …so on}

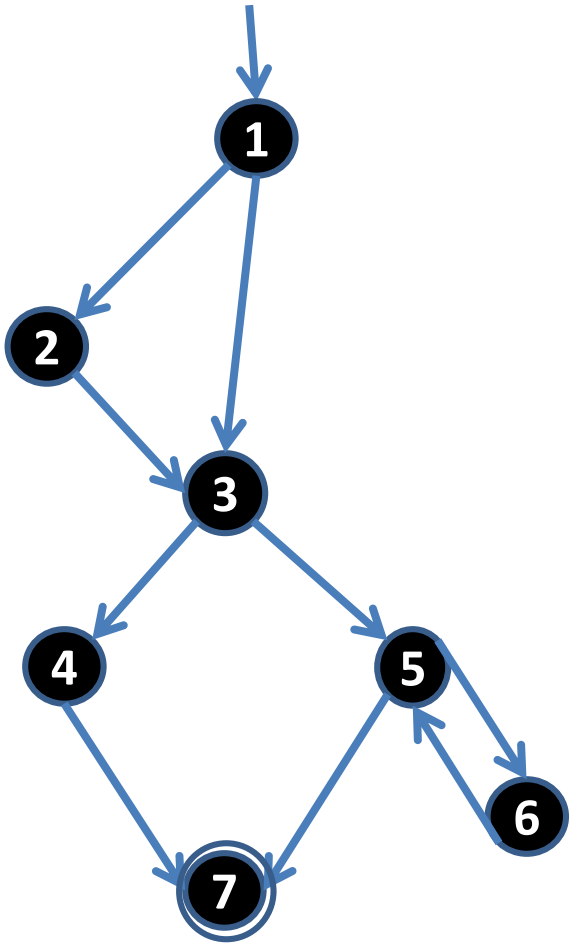


Complete and Specific Path Coverage

- A **graph having a loop, contains infinite number of paths**, and **complete path coverage is impossible**
- A **reasonable fair approach** can be **to give emphasis on loop boundary condition** and **skip repeated execution of the body**
- **Ideally, design test cases so that**
  - **Execute** the **loop** at is **boundary condition**: **skip the loop execution**
  - **Execute** the **loop once** with its normal execution
- This is **the idea behind prime path coverage**

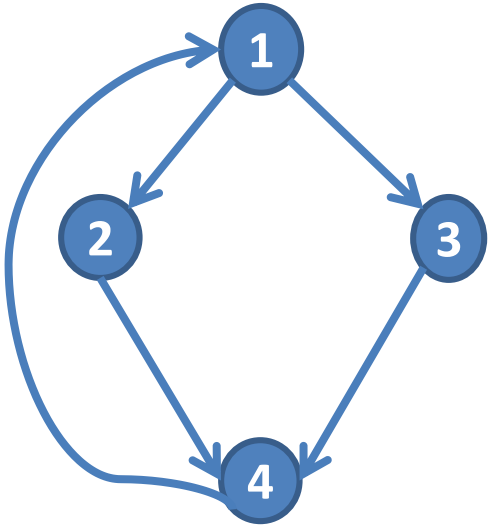
Simple Path and Prime Path in Graph

- **Simple Path** is **path** from node n_i to node n_j **if no node appears more than once except the first and last node**
  - **No internal loop allowed in simple path**
  - **A loop is a simple path**
  - **(1,3,5,6), (5,6,5), (1,2), and (2) are simple paths**
  - **(1,3,5,6,5,7) is NOT a simple path**
- **Prime path is a simple path that does not appear as a proper sub-path of any other simple path in the CFG.**



Prime Path in Graph

- **Prime path** is a **simple path** that **does not appear** as a **proper sub-path** of any other **simple path in the CFG.**
- **Steps for Finding Prime Paths:**
  - **Step -1**: Starting from length 0, find a set of all simple paths in the graph.
  - **Step -2**: Starting from length 0 delete paths from the set (generated in step 1) which are sub-paths of other paths.
  - **Step -3**: Leftover paths are the prime paths.

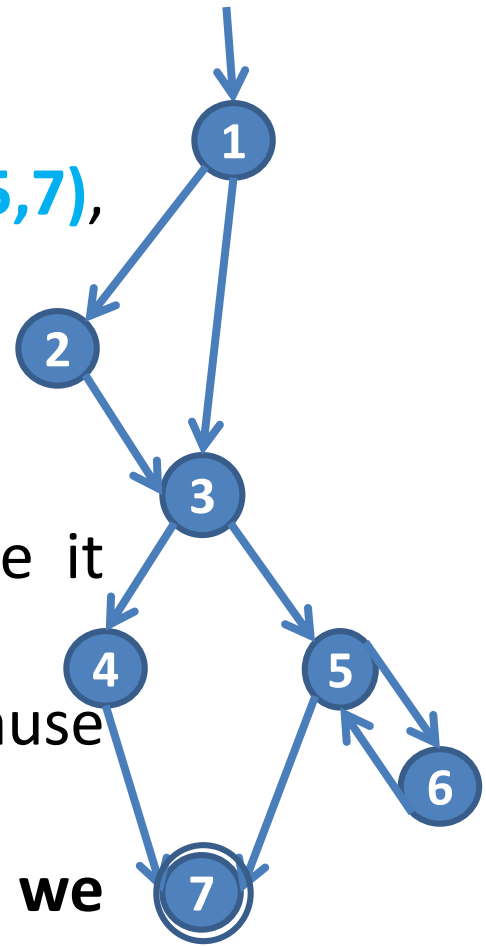


Finding Prime Path in Graph

- **Step 1: Starting from length 0, find a set of all simple paths in the graph**
  - **Simple Path of length 0: (1), (2), (3), (4)**
  - **Simple Path of length 1: (1,2), (1,3), (2,4), (3,4), (4,1)**
  - **Simple Path of length 2: (1,2,4), (1,3,4), (2,4,1), (3,4,1), (4,1,2), (4,1,3)**
  - **Simple Path of length 3: (1,2,4,1), (1,3,4,1), (2,4,1,2), (2,4,1,3), (3,4,1,2), (3,4,1,3), (4,1,2,4), (4,1,3,4)**
- **Step -2: Starting from length 0 DELETE paths generated in step 1, which are sub-paths of other larger length paths**
  - **Simple Path of length 0: ~~(1)~~, ~~(2)~~, ~~(3)~~, ~~(4)~~**
  - **Simple Path of length 1: ~~(1,2)~~, ~~(1,3)~~, ~~(2,4)~~, ~~(3,4)~~, ~~(4,1)~~**
  - **Simple Path of length 2: ~~(1,2,4)~~, ~~(1,3,4)~~, ~~(2,4,1)~~, ~~(3,4,1)~~, ~~(4,1,2)~~, ~~(4,1,3)~~**
  - **Simple Path of length 3: (1,2,4,1), (1,3,4,1), (2,4,1,2), (2,4,1,3), (3,4,1,2), (3,4,1,3), (4,1,2,4), (4,1,3,4)**
  - **Note: In this example, all paths except of length 3 are deleted, but that may not be always for all control flow graphs**
- **Step 3: The left over paths are prime paths. Here in total 8 prime paths are there: (2,4,1,2), (2,4,1,3), (1,3,4,1), (1,2,4,1), (3,4,1,2), (4,1,3,4), (4,1,2,4), (3,4,1,3)**

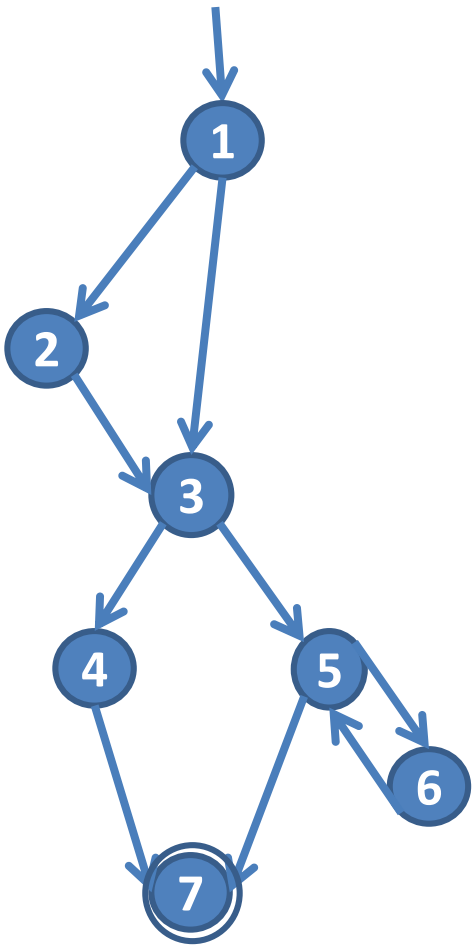
Finding Prime path

- **Prime path coverage** is **useful** for **designing test cases** for programs **having loops**.
- In the given CFG, there are **nine prime paths**
  - (5,6,5), **(6,5,6)**, (6,5,7), (1,3,4,7), **(1,3,5,7)**, (1,2,3,5,7), **(1,3,5,6)**, (1,2,3,4,7), (1,2,3,5,6)
- Out of these paths,
  - Path (1,3,5,7) skips the loop
  - Path (1,3,5,6) executes the loop once, because it executes node 6(body of loop) only once.
  - Path (6,5,6) executes loop more than once, because it executes node 6(body of loop) two times.
  - **If we design test cases, based on these 3 paths, we can check all functionalities of the loops**.



Finding Prime path

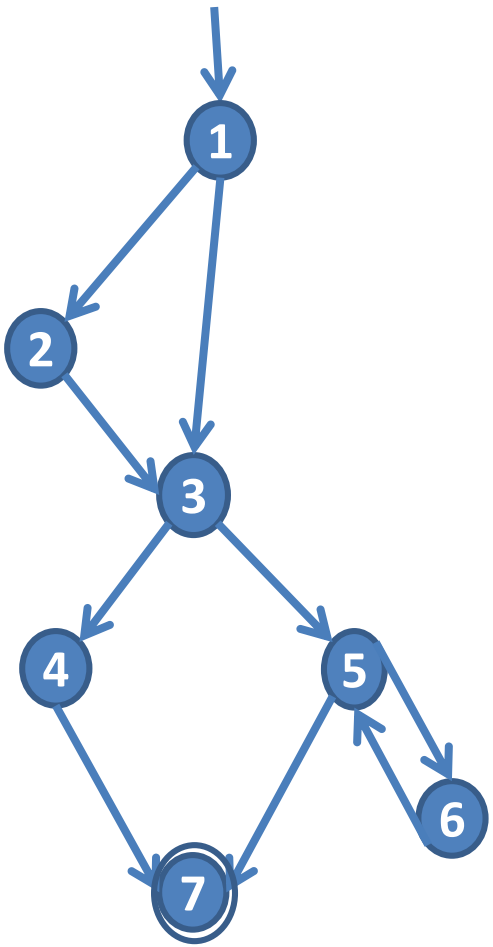
- **So, how to find** these **nine Prime paths** of the **control flow graph?**
- There is **no algorithms** as such for finding **prime paths.** However, you may **follow the three steps** discussed earlier for this. May be if you **fill the table given in the next slide column wise,** you can easily find out them.
- **You need not generate all the paths starting from length 0** as was done in last example.



Finding Prime paths

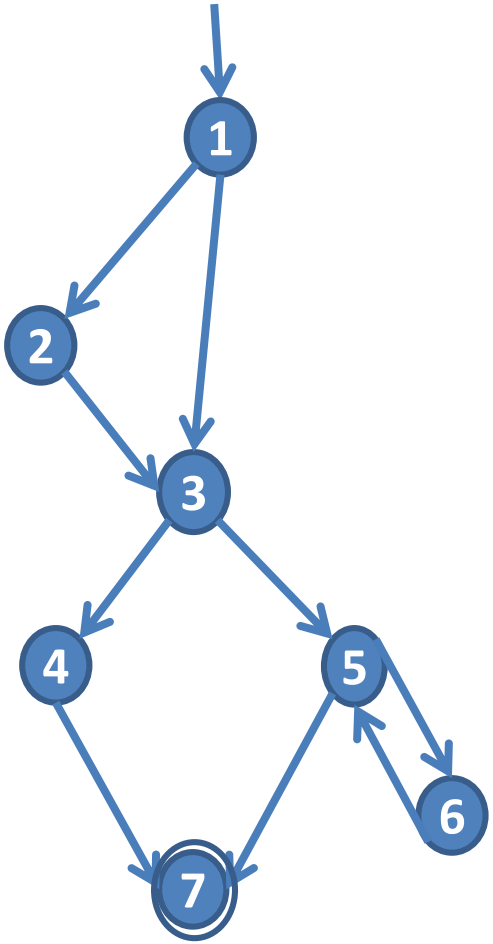
- **List out all the paths of length 0** in 1st Column L0. Mark (colour them in red) **those paths that reach the last node**, e.g. **node 7**. We can't extend these red paths any more.

L0					
1					
2					
3					
4					
5					
6					
7					



Finding Prime paths

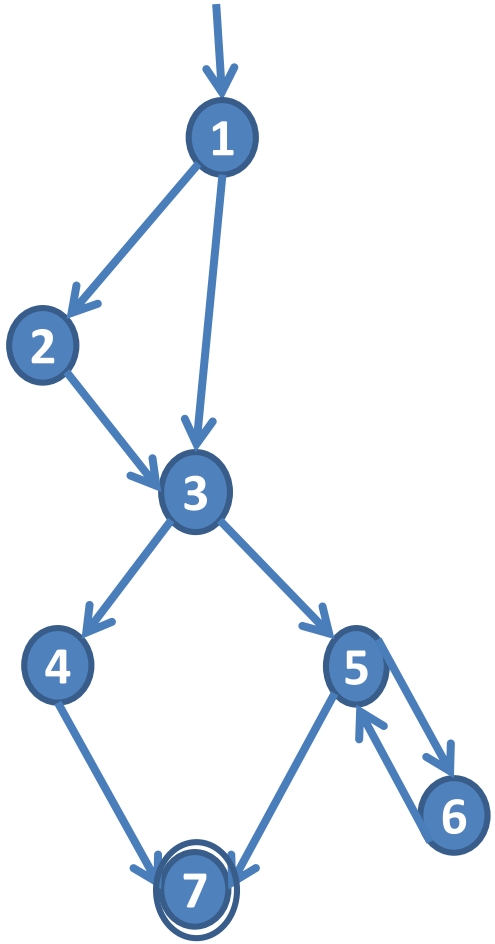
- **Extend** each of the L0 paths (non red) to make paths of length 1 (L1) and write down in 2nd **column L1**.
  - For example, first path consisting node 1 can be extended to create two paths, (1,2) and (1,3). Fifth path consisting node 5 can be extended to create two more paths, (5,6) and (5,7). Here path (5,7) can't be extended further and marked red.



L0	L1	L2	L3	L4	L5
1	12,13				
2	23				
3	34,35				
4	47				
5	56,57				
6	65				
7					

Finding Prime paths

- **Keep filling the table column wise**.
  - As you extend smaller path in to larger path, you may come across **another scenario**. If you extend the path by adding a new node, **prime path criteria will be violated**. For example, path (3,5,6) [column L2, row 3] can be extended to (3,5,6,5) but is not a prime path (2nd node 5 is repeated). Mark them also & do not extend.

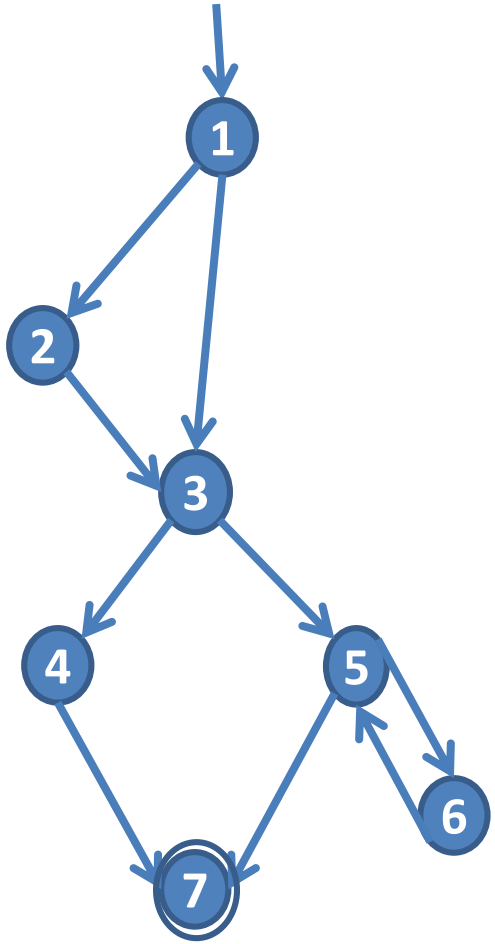


L0	L1	L2	L3	L4	L5
1	12,13	123,134,135			
2	23	234,235			
3	34,35	347,356,357			
4	47			Prime path rule violation	
5	56,57	565		End node reached	
6	65	656,657			
7					

Finding Prime paths

- **Ultimately** you will reach **in a column**, where you will **not able to generate any more larger paths**. **Table creation will be over** there.

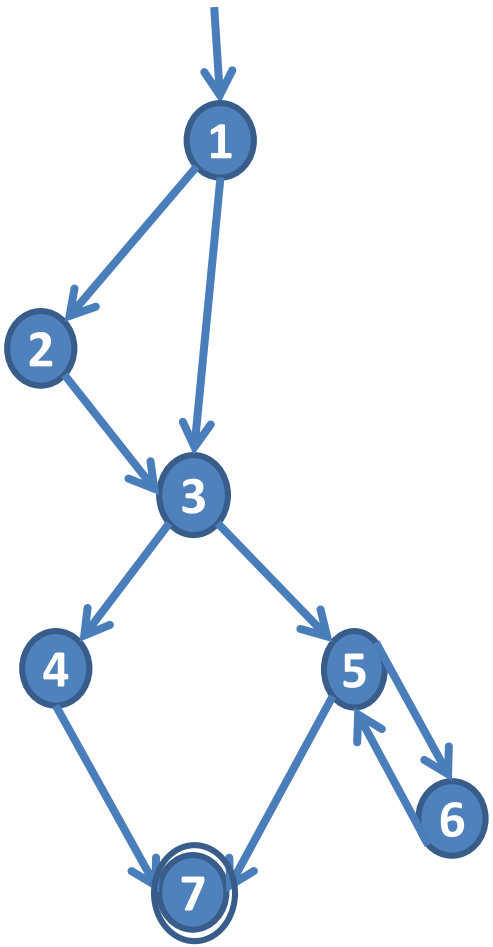
L0	L1	L2	L3	L4	L5
1	12,13	123,134,135	1234,1235,1347,1356,1357	12347,12356,12357	
2	23	234,235	2347,2356,2357		
3	34,35	347,356,357			
4	47			Prime path rule violation	
5	56,57	565		End node reached	
6	65	656,657			
7					



Finding Prime paths

L0	L1	L2	L3	L4	L5
1	12,13	123,134,135	1234,1235,1347 ,1356,1357	12347,12356,12357	
2	23	234,235	2347,2356,2357		
3	34,35	347,356,357			
4	47			Prime path rule violation	
5	56,57	565		End node reached	
6	65	656,657			
7					

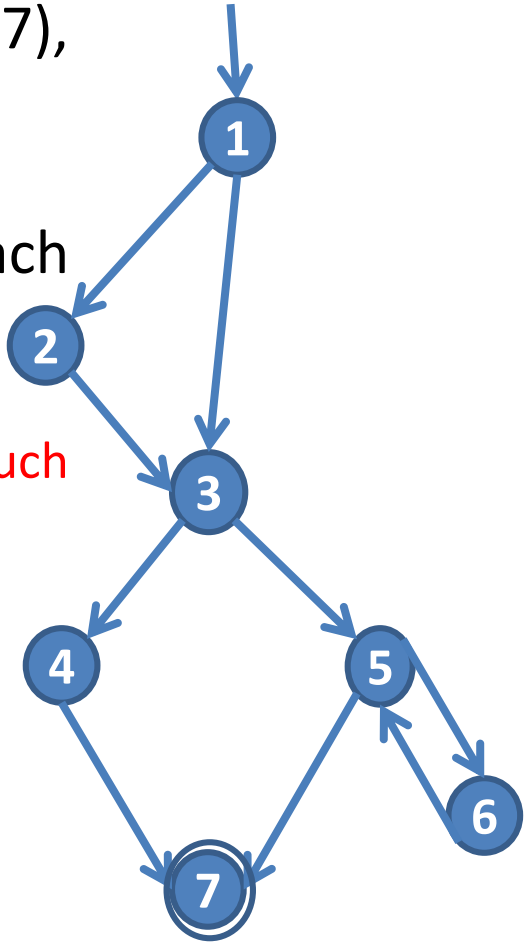
L0	L1	L2	L3	L4	L5
~~1~~	~~12,13~~	~~123,134,135~~	~~1234,1235~~,1347, 1356,1357	12347,12356,12357	
~~2~~	~~23~~	~~234,235~~	~~2347,2356,2357~~		
~~3~~	~~34,35~~	~~347,356,357~~			
~~4~~	~~47~~				
~~5~~	~~56,57~~	565		~~Sub-path removed~~	
~~6~~	~~65~~	656,657			
~~7~~					



Finding Test paths from the Prime paths

- **Nine prime paths:**
  - (~~5,6,5~~)*, (6,5,6), (~~6,5,7~~)*, (1,3,4,7), (1,3,5,7), (1,3,5,6), (1,2,3,4,7), (1,2,3,5,7), (1,2,3,5,6)
- **Finding Test Paths**
  - Start with the longer prime paths and extend each with the initial and final nodes in the graph
  - E.g.: Extend (1,2,3,5,6) as **(1,2,3,5,6,5,7)**

    *This extended path includes some other prime paths such as (5,6,5), (6,5,7). These are ignored further
  - No change required in the path **(1,2,3,5,7)**
  - No change required in the path **(1,2,3,4,7)**
  - Extend (1,3,5,6) as **(1,3,5,6,5,7)**
  - No change required in **(1,3,5,7)**
  - No change required in **(1,3,4,7)**
  - Extended (6,5,6) as **(1,3,5,6,5,6,5,7)**
    - **Or** Extend (1,3,5,6) as **(1,3,5,6,5,6,5,7)**

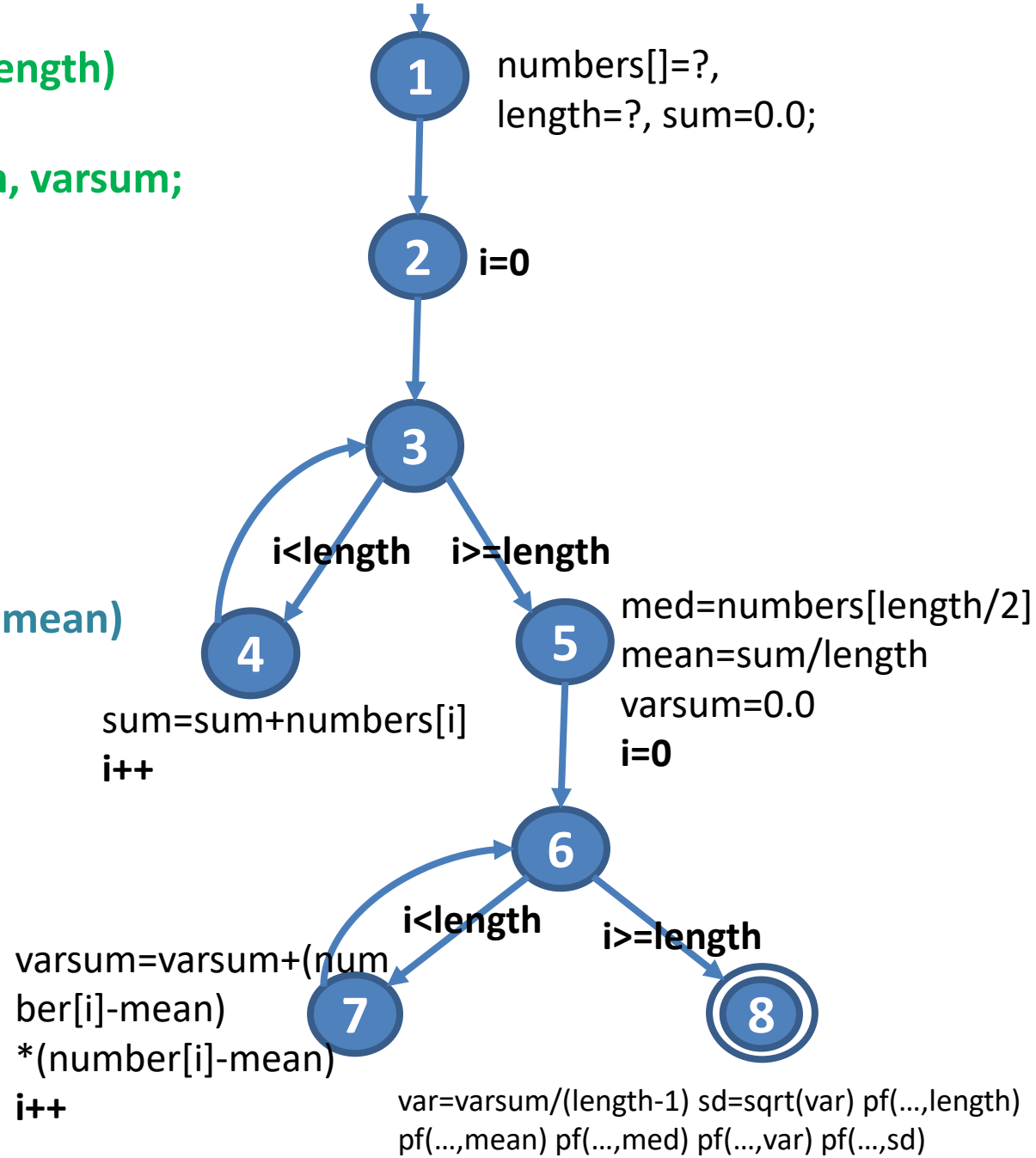


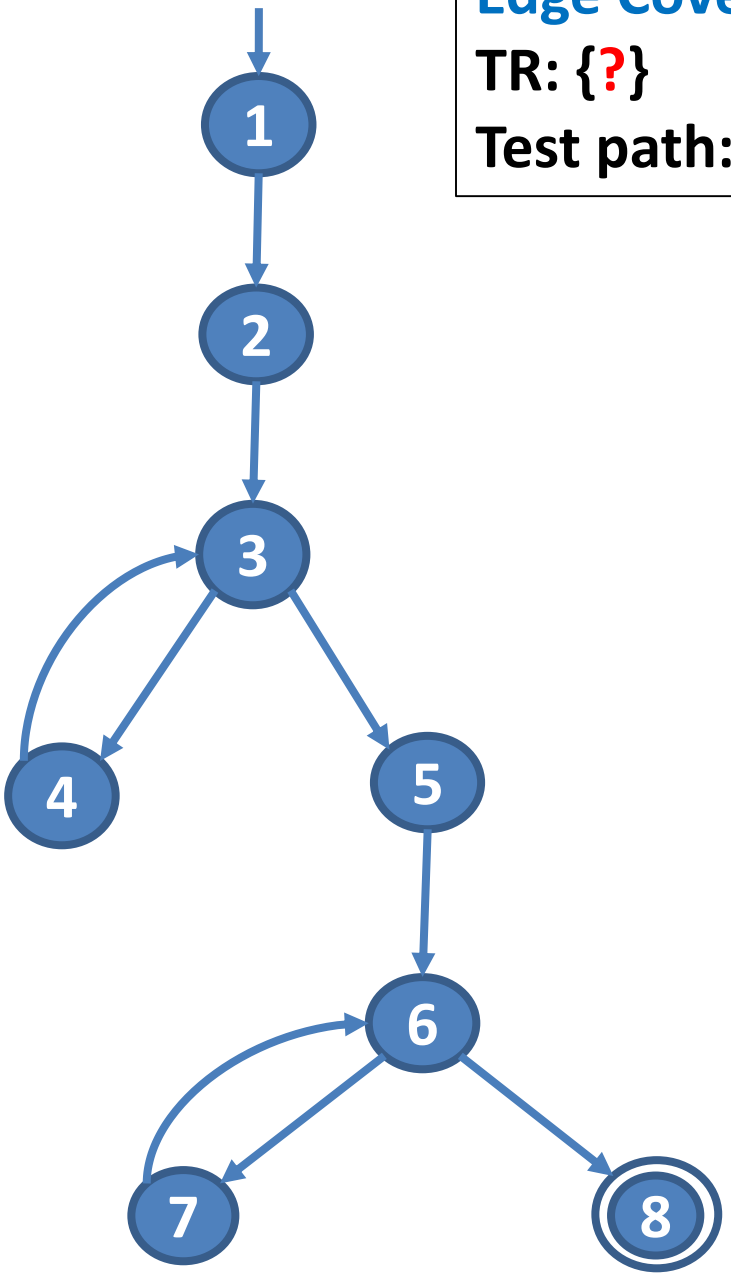
Finding Test paths from the Prime paths

- The set of test paths obtained **may not be the optimal one**.
  - Ensure that loops are skipped and also executed
  - The set can be further reduced. However for **practical software testing purpose the set of test paths obtained is acceptable**.

Another complete example

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%d",length);
  printf("Mean=%d",mean);
  printf("Median=%d",med);
  printf("Variance=%d",var);
  printf("Std. dev=%d",sd);
}
```





Edge Coverage:

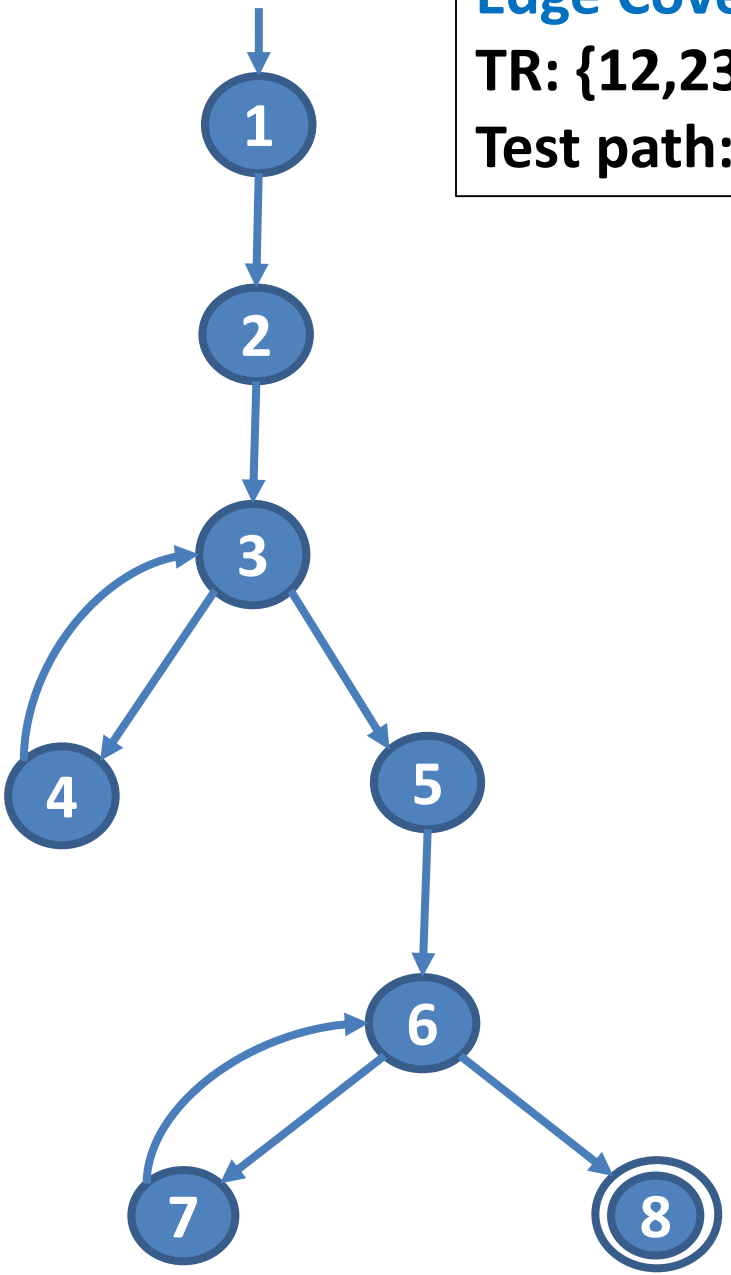
TR: {?}

Test path: {?}

Edge pair Coverage:

TR: {?}

Test path: {?}



Edge Coverage:

TR: {12,23,34,43,35,56,67,76,68}

Test path: {1234356768} Other test paths are also OK

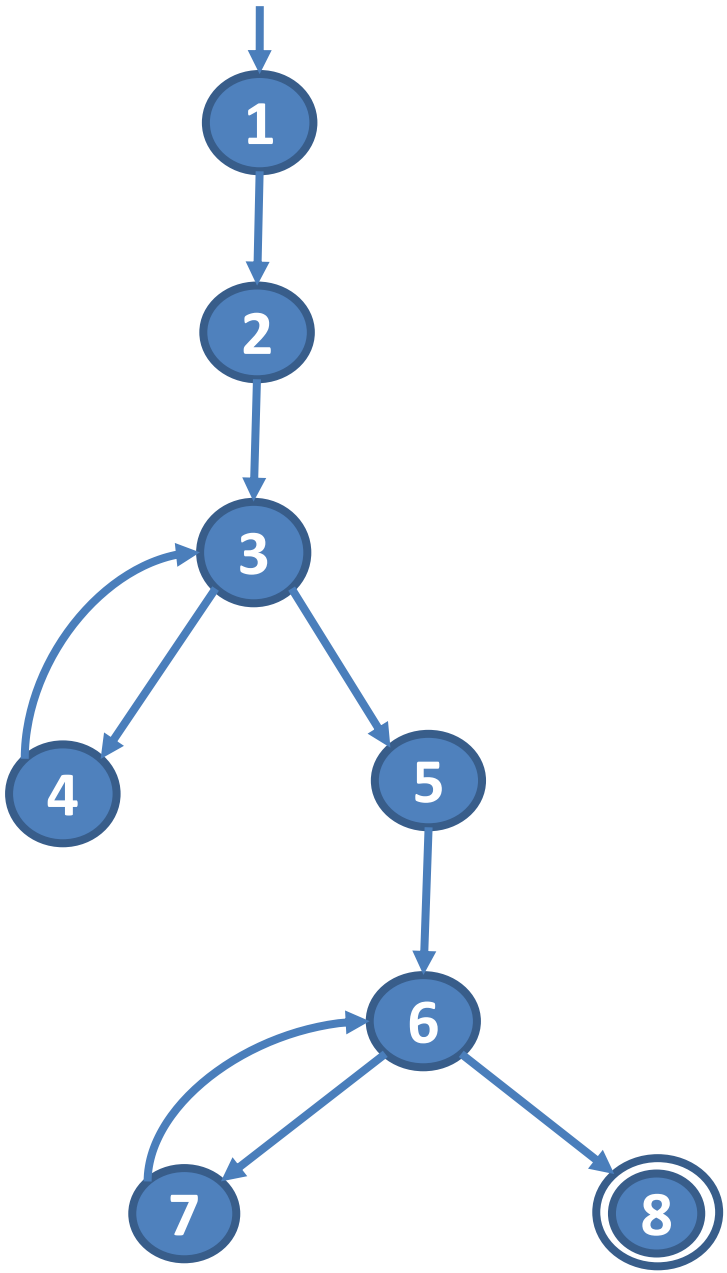
Edge pair Coverage:

TR: {

A.	**(1,2,3)**		
B.	**(2,3,4),**	**C.**	**(2,3,5)**
D.	**(3,4,3),**	**E.**	**(3,5,6)**
F.	**(4,3,5),**	**K.**	**(4,3,4)**
G.	**(5,6,7),**	**H.**	**(5,6,8)**
I.	**(6,7,6)**		
J.	**(7,6,8)**		
L.	**(7,6,7)**		

}

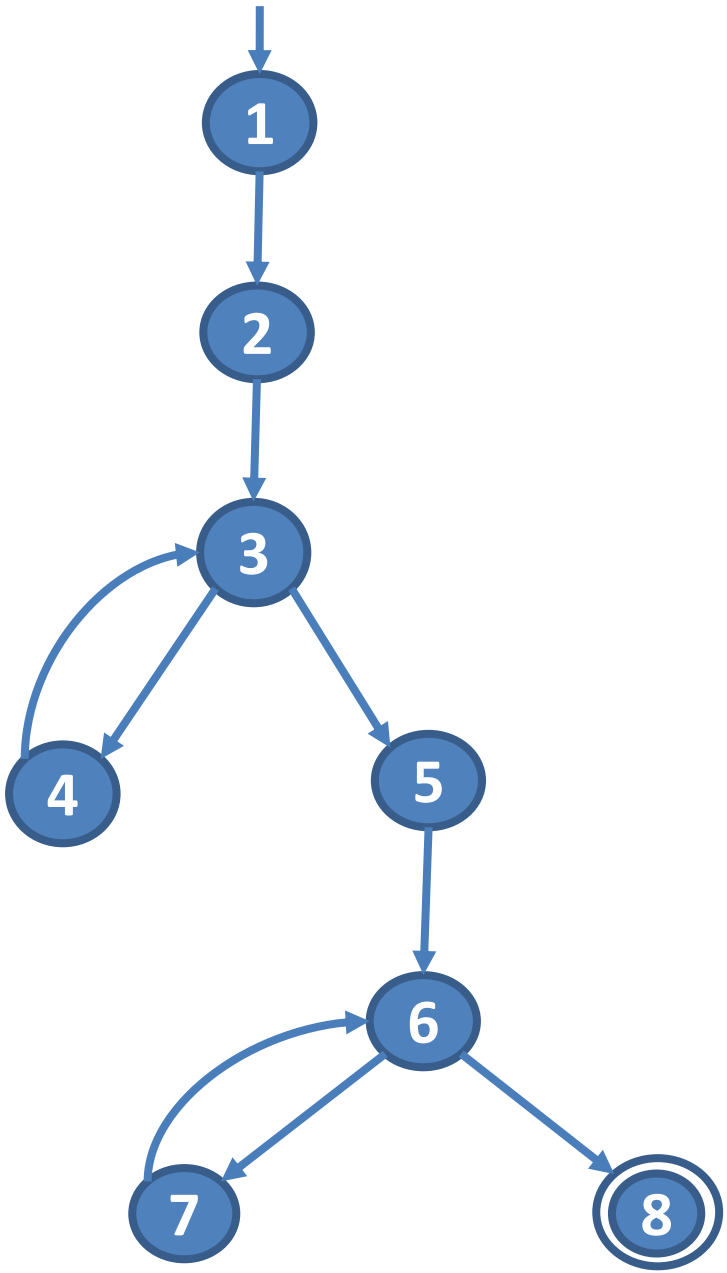
Test path: {1234356768}, {123568}, {1234343567678}



Prime path Coverage:

TR: {?}

Test paths: {?}



Prime path Coverage:

TR: {

A. (3,4,3)

B. (4,3,4)

C. (7,6,7)

D. (7,6,8)

E. (6,7,6)

F. (1,2,3,4)

G. (4,3,5,6,7)

H. (4,3,5,6,8)

I. (1,2,3,5,6,7)

J. (1,2,3,5,6,8)

}

Test paths: {1234356768}, {12343435676768}, {12343568}, {12356768}, {123568}

Round Trips

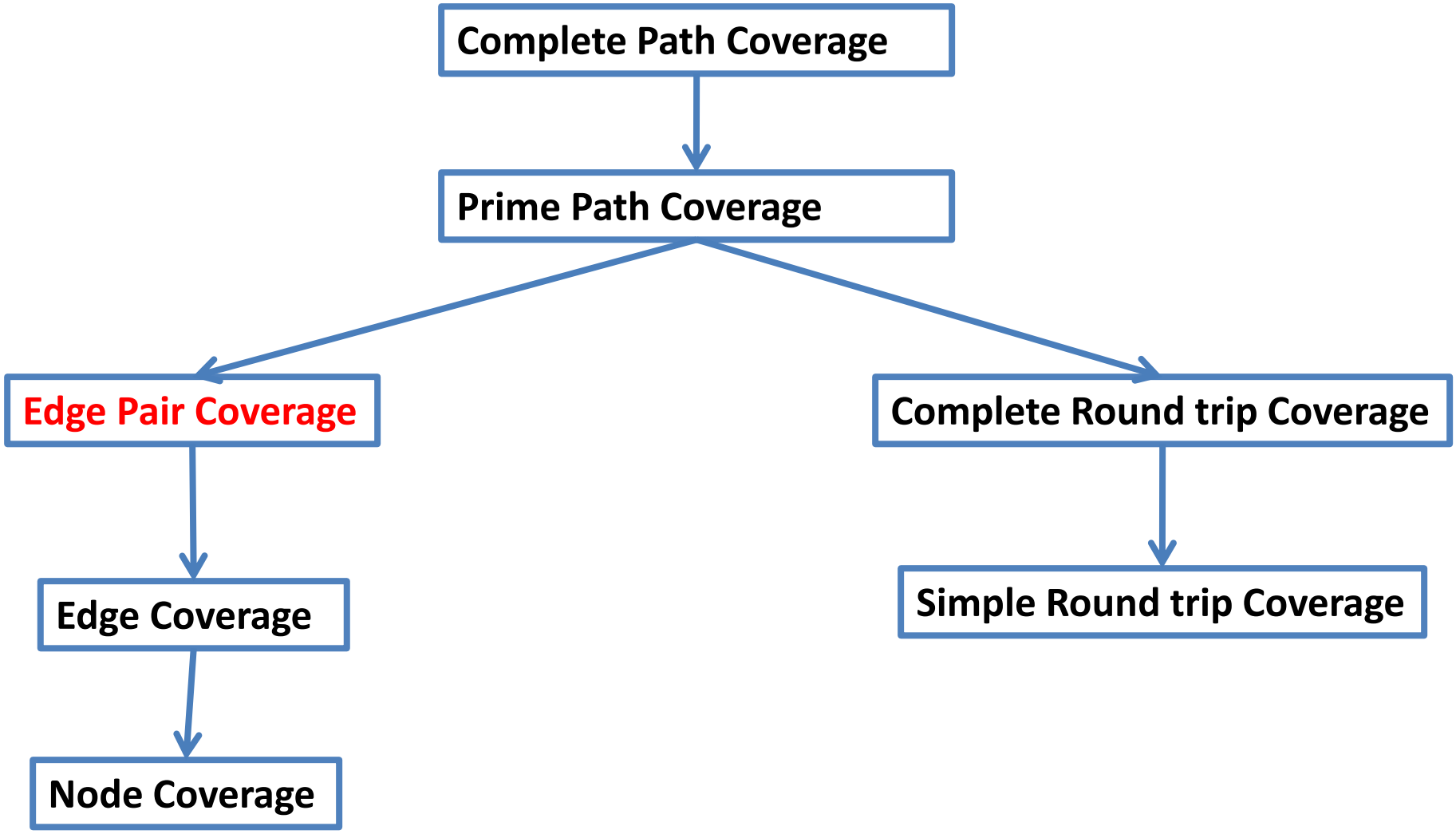
- **Round Trip path**: A **prime path** that **starts and ends at the same node**.
- **Simple round trip coverage**: TR contains at least one round trip path for each reachable node in G that begins and ends in a round trip path.
- **Complete round trip coverage**: TR contains all round trip paths for each reachable node in G.
- These criteria omit nodes and edges that are not in round trips.
  - Therefore, **they do not subsume edge-pair, edge and node coverage.**

Prime path coverage subsumes node coverage?
YES/NO

Prime path coverage subsumes edge coverage?
YES/NO

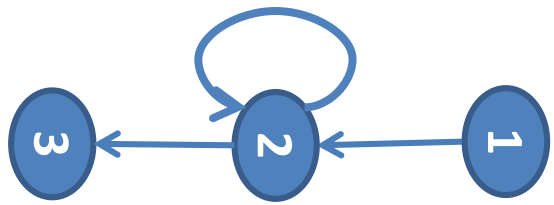
Prime path coverage subsumes edge-pair coverage?
YES/NO

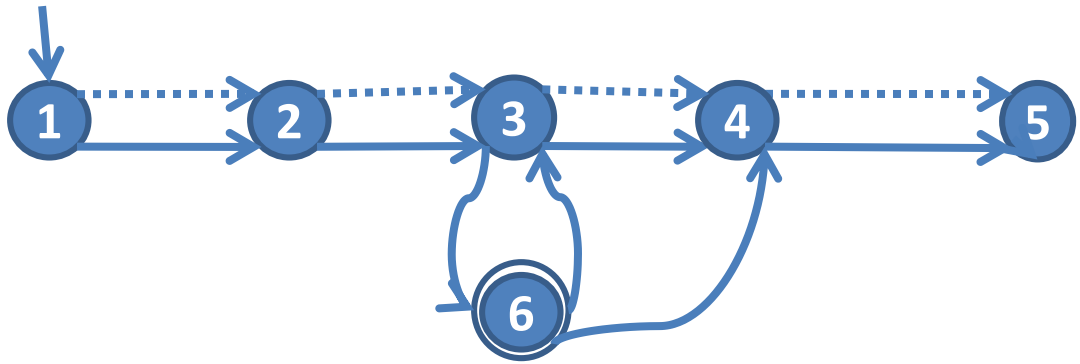
Structural Coverage Criteria



Prime Path coverage may not cover edge-pair

- **Prime path coverage**: TR contains each prime path in the graph G
  - May or may not cover edge-pair coverage
    - In a graph, where there are self loop, edge pair coverage requires the self loop to be visited
    - The **TR for the edge pair coverage** for the graph is **{(1,2,3),(1,2,2),(2,2,3),(2,2,2)}**
    - **TR for prime path** for the graph is **{(1,2,3),(2,2)}**



Special consideration in prime path implementation

- Prime paths by definition can't have internal loops.
- In many cases, it might be impossible to meet the test requirement of prime path coverage without internal loops
  - For example, do-while loop in C/C++ force execution of the loop at least once and that create a internal loop.
- That is the test paths that need prime path coverage TR need to have internal loops to make prime path coverage feasible.